Pre-contenido — Instrucciones y direccionamiento

Arquitectura de Computadores · Unidad 6 · Pre6-U6 Estudiante: González López

Parte 1 — Clasificación de instrucciones y la ISA x86

Pregunta 1 — Predicción de consecuencias

Si una actualización de microcódigo deshabilitara la caché de micro-operaciones, un bucle que se repite millones de veces tendría que obtener sus instrucciones decodificadas desde el front-end convencional con mayor frecuencia. En x86, las instrucciones pueden medir de 1 a 15 bytes y combinar prefijos, opcode, ModRM, SIB, desplazamientos e inmediatos. Identificar dónde termina una instrucción y traducirla a una o varias micro-operaciones demanda más trabajo que entregar micro-operaciones ya decodificadas desde una caché especializada. Por eso, cuando el bucle cabe en esa caché y el decodificador es el límite, perder su reutilización puede reducir el rendimiento y el IPC: el back-end podría recibir menos trabajo por ciclo.

El efecto no sería igual para todos los programas. En un bucle corto y caliente, la caché evita repetidamente el costo del decodificador. En un flujo lineal, ese ahorro se amortiza menos; si el programa está limitado por memoria o dependencias, deshabilitarla podría tener menor impacto que en uno limitado por decodificación. Los manuales de Intel muestran que el rendimiento también depende de la entrega de micro-operaciones, las predicciones y los recursos de ejecución (Intel, 2024a, 2024b).

Un procesador ARM no sufriría necesariamente una caída comparable: sus instrucciones A64 tienen longitud fija de 32 bits y una decodificación más regular (Arm, 2023). Aun así, también necesita buscar y decodificar instrucciones; una restricción hipotética equivalente en su front-end podría perjudicar bucles calientes. La diferencia es que su codificación fija simplifica delimitar instrucciones, no elimina el costo del front-end ni hace inmune a ARM a un cuello de botella de suministro.

Pregunta 2 — Reflexión crítica

Llamar CISC a un Intel Core o AMD Ryzen sigue siendo correcto si se especifica que se habla de la ISA visible: su contrato binario x86-64 ofrece instrucciones de longitud variable, muchos modos de direccionamiento y operaciones que pueden involucrar memoria. No describe por sí solo la organización interna del procesador. Los núcleos actuales decodifican instrucciones x86 en micro-operaciones, las renombran y las planifican para su ejecución superescalar y fuera de orden. Por tanto, la oposición CISC/RISC es útil para comparar interfaces y objetivos históricos, pero insuficiente para deducir directamente el comportamiento interno de un chip moderno (Intel, 2024a; AMD, 2024).

Para quien programa en ensamblador, esto significa que “una instrucción CISC hace más por ciclo” no es una regla confiable. Una instrucción de memoria y operación puede traducirse a varias micro-operaciones, mientras que ciertas secuencias cortas pueden fusionarse o aprovechar rutas optimizadas. El costo depende del modelo concreto: latencia, rendimiento de emisión, puertos disponibles, dependencias, predicción de saltos y aciertos de caché. Por ejemplo, una instrucción que combina

carga y suma puede ocupar menos bytes que una carga y una suma separadas, pero eso no garantiza menor latencia ni más rendimiento para cualquier CPU.

La estrategia responsable es partir de las garantías de la ISA y luego consultar guías o mediciones para la microarquitectura objetivo. Los tiempos y la descomposición pueden variar entre procesadores (Intel, 2024a, 2024b; Fog, 2025). La ISA describe qué calcula el programa; para explicar cuánto tarda, también se requieren la microarquitectura, el código generado y la carga. CISC describe sobre todo el contrato externo, no predice por sí solo el rendimiento.

Pregunta 3 — Síntesis

Para recorrer dos arreglos de números de punto flotante y producir un tercero puede definirse `z[i] = x[i] + y[i]`. Si el segundo arreglo se usa como destino, `y[i] = 1*x[i] + y[i]` es SAXPY con coeficiente uno. En una versión escalar, las instrucciones de transferencia cargan cada elemento, la suma calcula el resultado, otra transferencia lo almacena y un salto actualiza el índice hasta terminar. En una versión AVX, cada registro YMM contiene ocho valores `float` de 32 bits: `vmovups` carga ocho elementos de `x`, `vaddps` suma ocho de `y`, y `vmovups` almacena ocho resultados. Un contador avanza ocho posiciones por iteración y un tramo final escalar gestiona los elementos restantes. Esta versión requiere que el procesador y el sistema operativo habiliten el estado AVX correspondiente; el compilador también puede producirla mediante auto-vectorización cuando las dependencias y las condiciones del bucle lo permiten (Intel, 2024a, 2024b).

Los tres elementos cumplen funciones distintas. Transferencias y control describen cómo localizar arreglos, repetir el bucle y guardar resultados. SIMD aumenta el paralelismo de datos al efectuar varias sumas independientes por instrucción vectorial. La traducción a micro-operaciones y la caché de micro-operaciones son mecanismos internos del núcleo: las instrucciones vectoriales y el control se decodifican y sus micro-operaciones pueden reutilizarse cuando el bucle vuelve a ejecutarse. Ninguno sustituye a los otros: SIMD no elige por sí mismo el siguiente bloque de memoria, y la caché de micro-operaciones no convierte operaciones escalares en vectoriales.

Sin auto-vectorización, el compilador ejecutaría más iteraciones e instrucciones escalares de carga, suma y almacenamiento. La mejora real depende del ancho de banda, alineación, dependencias, frecuencia AVX y tamaño del arreglo; ocho valores por instrucción no garantizan ocho veces más velocidad. Se debe medir la carga y revisar el código generado antes de atribuir una aceleración a SIMD (Intel, 2024b; Fog, 2025).

Parte 2 — Modos de direccionamiento

Pregunta 1 — Análisis comparativo

En x86-64, una referencia absoluta identifica una dirección fija o lleva un desplazamiento que el enlazador resuelve; una referencia RIP-relativa calcula `dirección efectiva = RIP de la siguiente instrucción + desplazamiento relativo con signo de 32 bits`. Por ejemplo, `lea rax,[rel tabla]` obtiene la dirección de `tabla` sin leer su contenido, y `mov eax,[rel tabla]` carga sus primeros cuatro bytes. El desplazamiento relativo permite que el código y sus datos cercanos se trasladen juntos: al cambiar la base, la distancia interna no cambia. Por eso encaja con ejecutables PIE y bibliotecas sometidos a ASLR. En cambio, una dirección absoluta codificada en el binario puede requerir una reubicación o una

tabla de relocaciones; no es siempre imposible con ASLR, pero suele ser menos conveniente para código compartido y relocalizable (AMD, 2024; Intel, 2024a).

El relativo a RIP no alcanza cualquier punto de los 64 bits: su desplazamiento de 32 bits limita el destino a aproximadamente ±2 GiB. Además, x86-64 no ofrece con ModRM una dirección absoluta arbitraria de 64 bits como forma general de `mov`. La dirección absoluta conserva usos legítimos, por ejemplo, registros de dispositivos mapeados a memoria en firmware con un mapa de hardware fijo.

RIP es el puntero de instrucción de 64 bits y la base implícita de este modo; no es un registro de propósito general que se use libremente como índice. En modo de 32 bits EIP existe, pero la forma RIP-relative descrita aquí es propia del modo de 64 bits. Por ello, un curso que solo ensambla `.COM` de 16 bits no puede demostrarla directamente: debe añadir un ejemplo NASM `BITS 64` para explicar y verificar esta codificación por separado (AMD, 2024; Intel, 2024a).

Pregunta 2 — Toma de decisión con justificación

Para calcular `EBX + ECX4 + 8` *en una función identificada como crítica mediante perfilado, la elección inicial sería* `lea eax,[ebx + ecx4 + 8]`. LEA calcula la dirección efectiva indicada por una expresión de base, índice, escala y desplazamiento, pero no lee la memoria apuntada. En este caso usa EBX como base, ECX como índice, una escala permitida de cuatro y un desplazamiento de ocho. Conserva los registros fuente y no cambia EFLAGS; una secuencia aritmética que utilice `imul` y `add` normalmente sí cambia flags y requiere instrucciones adicionales. La operación también debe interpretarse en el ancho de destino: escribir EAX trunca a 32 bits, mientras que LEA con destino RAX preservaría el resultado de 64 bits en modo de 64 bits.

LEA no reemplaza una multiplicación arbitraria: la escala solo puede ser 1, 2, 4 u 8 y la suma debe ajustarse a la codificación. Tampoco carga el dato apuntado. Usarla para matemáticas poco evidentes puede confundir al lector y dificultar el mantenimiento; conviene explicar la intención. El rendimiento varía según microarquitectura, por lo que la forma más compacta no siempre es más rápida (Intel, 2024a, 2024b; Fog, 2025).

La decisión se justifica si el perfilado confirma que esa función es un cuello de botella, el cálculo aparece con frecuencia y las mediciones en el procesador objetivo muestran una mejora sin perjudicar otras dependencias. De no existir evidencia de ese costo, debe preferirse primero la forma más legible. Incluso en código crítico se documenta el propósito de LEA como suma escalada, se verifica que los flags no sean necesarios y se mantiene una prueba de equivalencia. Así se aprovecha el modo indexado sin asumir que una sola instrucción siempre significa menor latencia.

Pregunta 3 — Investigación de casos

En la convención CDECL de 32 bits, el llamador suele apilar los argumentos de derecha a izquierda y ejecuta `call`, que apila la dirección de retorno. Al entrar, la función ejecuta `push ebp` y `mov ebp,esp`. En ese instante `[ebp]` contiene el EBP anterior, `[ebp+4]` la dirección de retorno y `[ebp+8]` el primer argumento; el segundo argumento está en `[ebp+12]`. Si reserva ocho bytes con `sub esp,8`, las variables locales pueden ubicarse en `[ebp-4]` y `[ebp-8]`. Un ejemplo suma dos enteros y guarda temporalmente resultado y copia:

```
; int sumar_y_guardar(int a, int b)
sumar_y_guardar:
    push ebp
```

```
    mov  ebp, esp
    sub  esp, 8
    mov  eax, [ebp+8]
    add  eax, [ebp+12]
    mov  [ebp-4], eax
    mov  dword [ebp-8], 0
    mov  eax, [ebp-4]
    mov  esp, ebp
    pop  ebp
    ret
```

EBP permanece fijo aunque ESP cambie por temporales, llamadas o reservas adicionales; por eso ofrece desplazamientos estables para argumentos y variables. El depurador puede seguir la cadena de EBP guardados para reconstruir marcos y obtener direcciones de retorno, siempre que el código conserve marcos convencionales. Optimización que omite frame pointers, corrupción de pila o convenciones diferentes pueden romper esa cadena; los sistemas modernos también usan metadatos de unwind para reconstruir marcos con mayor fiabilidad (System V ABI, 1997; Intel, 2024a).

La respuesta alinea offsets de palabra con el prólogo exacto: después de reservar ocho bytes, `[ebp-4]` y `[ebp-8]` ocupan memoria local; al establecer `esp=ebp` se descartan ambas y `pop ebp` restaura el valor previo. En ensamblador de 16 bits se suele usar BP con palabras de dos bytes y `ret` según los parámetros; no se deben mezclar esos offsets con el ejemplo de 32 bits. Así, Base+Desplazamiento hace explícita la relación entre el marco y sus campos, pero el depurador necesita además símbolos o información de unwind para mostrar nombres de variables.

Referencias

AMD. (2024). *AMD64 Architecture Programmer's Manual, Volume 1: Application Programming* (Publication No. 24592). https://www.amd.com/content/dam/amd/en/documents/processor-tech-docs/programmer-references/24592.pdf

Agner Fog. (2025). *Instruction tables: Lists of instruction latencies, throughputs and micro-operation breakdowns for Intel, AMD and VIA CPUs*. https://www.agner.org/optimize/instruction_tables.pdf

Arm. (2023). *Arm Architecture Reference Manual for A-profile architecture* (Document No. DDI 0487). https://developer.arm.com/documentation/ddi0487/latest

Fog, A. (2025). *The microarchitecture of Intel, AMD and VIA CPUs: An optimization guide for assembly programmers and compiler makers*. https://www.agner.org/optimize/microarchitecture.pdf

Intel. (2024a). *Intel 64 and IA-32 architectures software developer's manual*. https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html

Intel. (2024b). *Intel 64 and IA-32 architectures optimization reference manual*. https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html

System V Application Binary Interface. (1997). *Intel386 architecture processor supplement* (4th ed.). https://refspecs.linuxfoundation.org/elf/abi386-4.pdf